



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME: COMPUTER ORGANIZATION AND ARCHITECTURE

SUBJECT CODE: SECR 1033

SEMESTER: 2 - 2023/24

LAB TITLE: Programming 1: Assembly Language Fundamentals

INSTRUCTION: Student is required to have instructor's signature before proceed from each lab work.

Name: 1. YASMIN BATRISYIA BINTI ZAHIRUDDIN

STUDENT INFO : 2. ANIS SAFFIYA BINTI JANAI

Metric No: 1. A23CS0201

2. A23CS0049

Section: 03

Email: _____

SUBMITTED DATE: _____

COMMENTS:

Lab # 1

Simple program to familiarize with Program Code, Rebuild & Start Without Debugging

Execute the programs below:

PART 1:

i. Part A: Adding and Subtracting Integers

```
TITLE Add and Subtract (AddSub.asm)

; This program adds and subtracts 32-bit integers
; Authors:
; Date:
; Revision:

INCLUDE Irvine32.inc

.data
TOTAL dword 0 ; a variable named TOTAL (declared as DWORD)

.code

main PROC

    mov eax, 123400h ; Set EAX with the value of 123400h
    add eax, 567800h ; Add the content of EAX with 567800h
    sub eax, 77700h ; Subtract content of EAX with 77700h
    mov TOTAL, eax ; Store content of EAX to TOTAL

    call DumpRegs

    exit
main ENDP

END main
```

Experimental Results:

a) Rebuild & Start Without Debugging



Completed *Fully* *Partially* : Checked by: _____

Screenshot Result:

```
EAX=00613500 EBX=009C4000 ECX=00CC10AA EDX=00CC10AA
ESI=00CC10AA EDI=00CC10AA EBP=00AFFB80 ESP=00AFFB74
EIP=00CC3679 EFL=00000206 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=1

C:\Users\ybzah\source\repos\lab 1 new\Debug\lab 1 new.exe (process 8776) exited with code 0.
Press any key to close this window . . .
```

ii. Part B: Adding Variables

```
TITLE Add and Subtract, Version 2 (AddSub2.asm)

; This program adds and subtracts 32-bit unsigned
; integers and stores the sum in a variable.
; Authors:
; Date:
; Revision:

INCLUDE Irvine32.inc
.data
val1 DWORD 10000h
val2 DWORD 40000h
val3 DWORD 20000h
finalVal DWORD ?
.code

main PROC
mov eax,val1      ; start with 10000h
add eax,val2      ; add 40000h
sub eax,val3      ; subtract 20000h
mov finalVal,eax ; store the result (30000h)

call DumpRegs    ; display the registers

exit

main ENDP
END main
```

Experimental Results:

a) Rebuild & Start Without Debugging



Completed *Fully* *Partially* : Checked by: _____

Screenshot Result:

```
Microsoft Visual Studio Debug Console
EAX=00030000 EBX=0092A000 ECX=00C210AA EDX=00C210AA
ESI=00C210AA EDI=00C210AA EBP=00AFF8BC ESP=00AFF8B0
EIP=00C2367B EFL=00000206 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=1

C:\Users\ybzah\source\repos\lab 1 new\Debug\lab 1 new.exe (process 11740) exited with code 0.
Press any key to close this window . . .
```

iii. Part C: Add and Subtract 8 and 16-Bit Version

```
TITLE Add and Subtract, Version 3
; This program adds and subtracts 8 and 16 bit
; unsigned integers and stores the sum in a variable.
; Authors:
; Date:
; Revision:

INCLUDE Irvine32.inc

.data
valw1 WORD 1000h
valw2 WORD 4000h
valw3 WORD 2000h
finalValw WORD ?

valb1 BYTE 10h
valb2 BYTE 40h
valb3 BYTE 20h
finalValb BYTE ?

.code
main PROC
mov ax,valw1; start with 10000h
add ax,valw2      ; add 40000h
sub ax,valw3      ; subtract 20000h
mov finalValw,ax  ; store the result (30000h)
call DumpRegs    ; display the registers

mov ah,valb1; start with 10000h
add ah,valb2      ; add 40000h
sub ah,valb3      ; subtract 20000h
mov finalValb,ah  ; store the result (30000h)
call DumpRegs    ; display the registers

exit
main ENDP
END main
```

Experimental Results:

a) Rebuild & Start Without Debugging



Completed *Fully* *Partially* : Checked by: _____

Screenshot Result:

```
Microsoft Visual Studio Debu:  x  +  v

EAX=012F3000  EBX=011B8000  ECX=006410AA  EDX=006410AA
ESI=006410AA  EDI=006410AA  EBP=012FFA3C  ESP=012FFA30
EIP=0064367F  EFL=00000206  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=1

EAX=012F3000  EBX=011B8000  ECX=006410AA  EDX=006410AA
ESI=006410AA  EDI=006410AA  EBP=012FFA3C  ESP=012FFA30
EIP=0064369C  EFL=00000206  CF=0  SF=0  ZF=0  OF=0  AF=0  PF=1

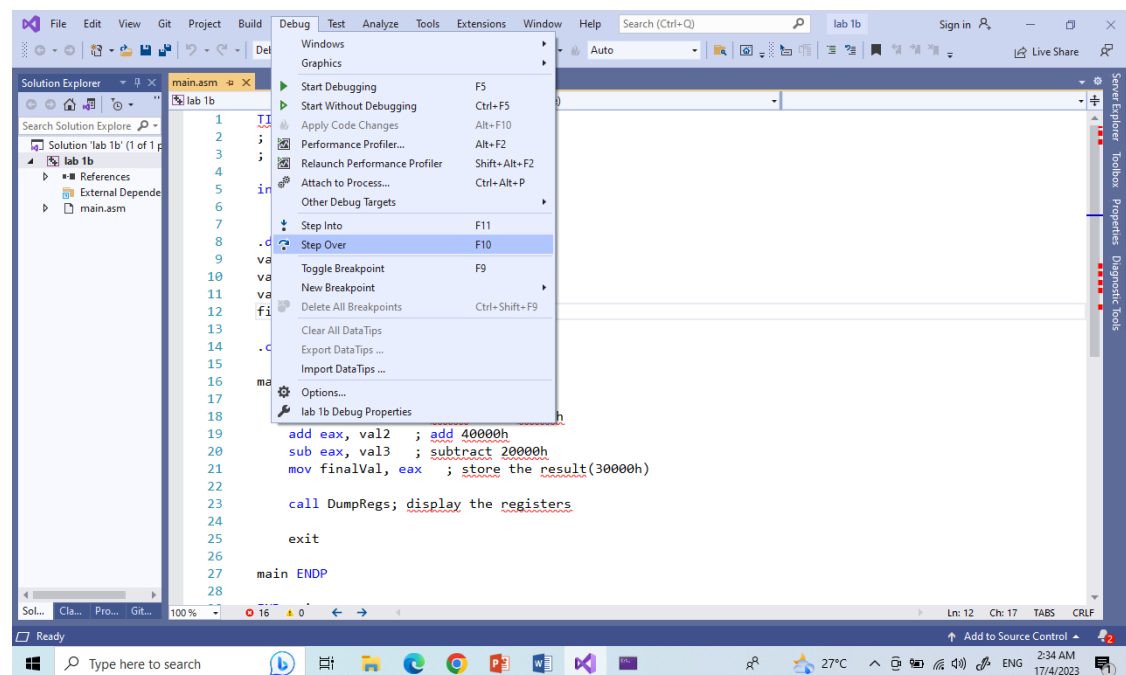
C:\Users\ybzah\source\repos\lab 1 new\Debug\lab 1 new.exe (process 1424) exited with code 0.
Press any key to close this window . . .
```

Detail Debugging Process

PART 2:

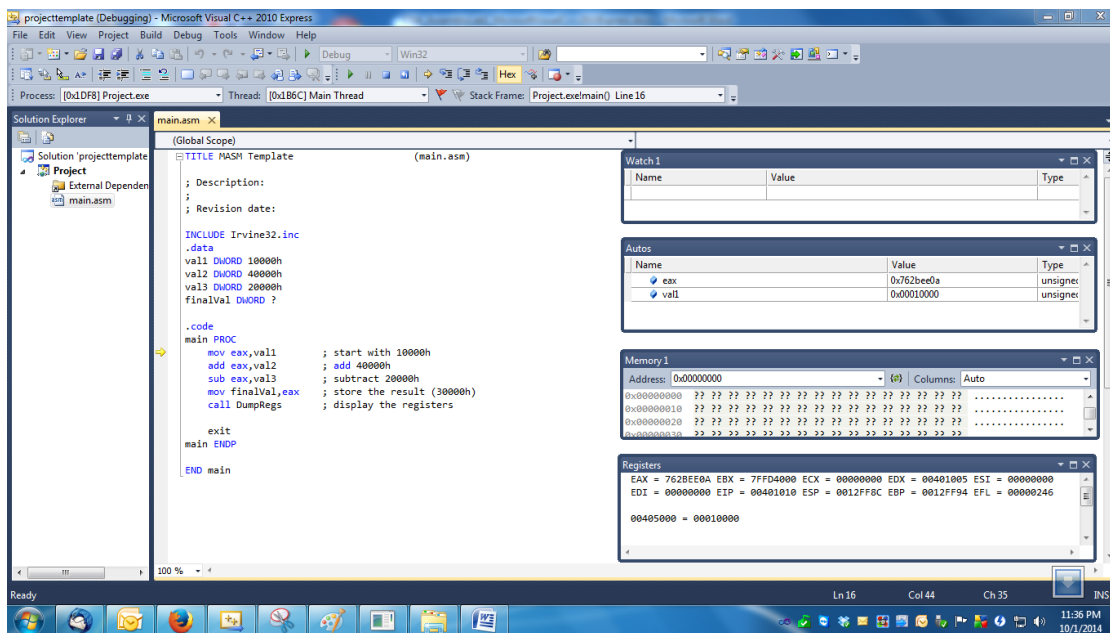
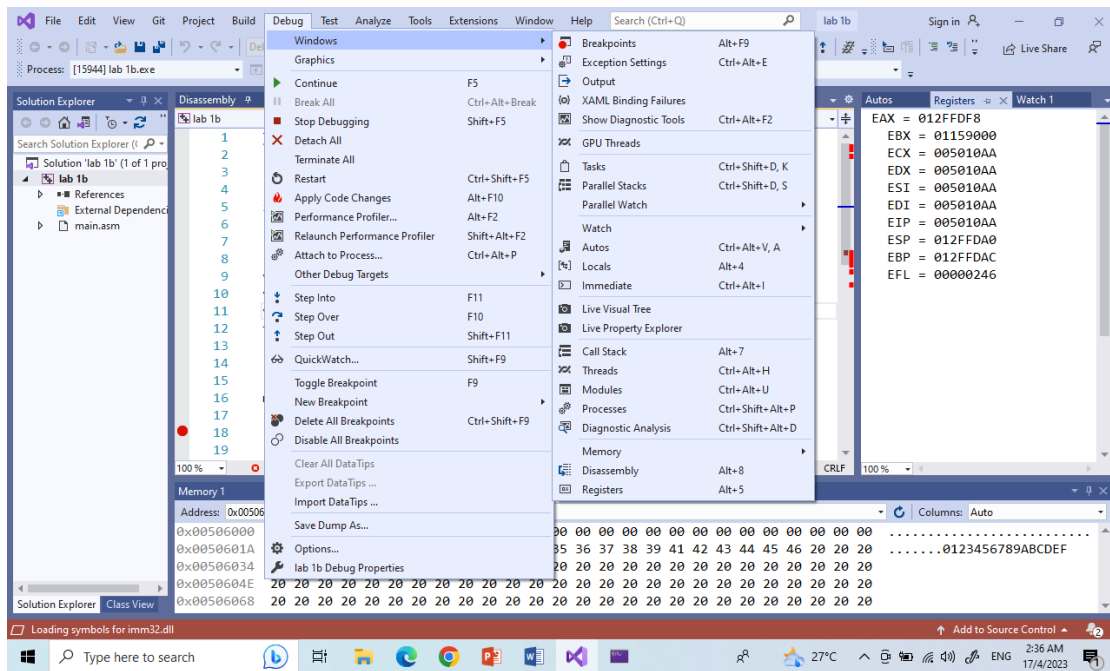
Run Debugging Process for Part B, Part C and Capture Video

1. Press F10 for step by step debugging.

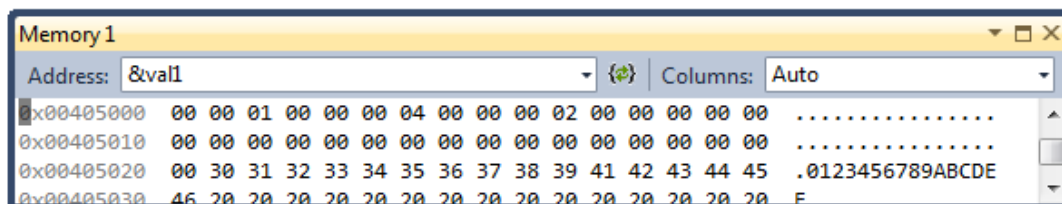


3. Open 5 windows:

- Watch
- Autos
- Memory
- Registers
- Disassembly



4. Default memory data segment value is at first variable: &val1 or 0x00405000 (depend on memory used).



5. Add variables val1, val2, val3 and finalval to Watch

Watch 1		
Name	Value	Type
val1	0x00010000	unsigned long
val2	0x00040000	unsigned long
val3	0x00020000	unsigned long
finalVal	0x00000000	unsigned long

6. F10 to trace/step the assembly program line by line.

7. Please debug by looking at value changes at Watch, Memory, Autos and Registers windows.

8. Disassembly window can show the following optional information:

- Memory address where each instruction is located. For native applications, it is the actual memory address. For Visual Basic or C#, it's an offset from the beginning of the function.
- Source code from which the assembly code derives.
- Code bytes, that is, the byte representations of the actual machine or MSIL instructions.
- Symbol names for the memory addresses.
- Line numbers corresponding to the source code.

What is Byte Code (or Machine Code) for the following assembly instructions:

```

mov ax, valw1 ; start with 10000h
add ax, valw2 ; add 40000h
sub ax, valw3 ; subtract 20000h
mov finalValw, ax ; store the result (30000h)

```

Byte Code:

```

Disassembly
Address: main(void)
Viewing Options
[ ] Show code bytes [x] Show address
[x] Show source code [x] Show symbol names
[x] Show line numbers
008F365F CC int 3
--- C:\Users\ybzah\source\repos\lab 1c\lab 1c\main.asm -----
22: mov ax, valw1; start with 10000h
008F3660 66 A1 00 60 8F 00 mov ax, word ptr [valw1 (08F6000h)]
23: add ax, valw2; add 40000h
008F3666 66 03 05 02 60 8F 00 add ax, word ptr [valw2 (08F6002h)]
24: sub ax, valw3; subtract 20000h
008F366D 66 28 05 04 60 8F 00 sub ax, word ptr [valw3 (08F6004h)]
25: mov finalValw, ax; store the result (30000h)
008F3677 66 A3 06 60 8F 00 mov word ptr [finalValw (08F6006h)], ax
26: call DumpRegs; display the registers
008F367A EB 62 DA FF FF call _DumpRegs@0 (08F10E1h)
27:
28: mov ah, valb1; start with 10000h
008F367F 8A 25 09 60 8F 00 mov ah, byte ptr [valb1 (08F6008h)]
29: add ah, valb2; add 40000h
008F3685 02 25 09 60 8F 00 add ah, byte ptr [valb2 (08F6009h)]
30: sub ah, valb3; subtract 20000h
008F368B 2A 25 0A 60 8F 00 sub ah, byte ptr [valb3 (08F600Ah)]
31: mov finalValb, ah; store the result (30000h)
008F3691 88 25 09 60 8F 00 mov byte ptr [finalValb (08F600Bh)], ah
32: call DumpRegs; display the registers
008F3697 EB 45 DA FF FF call _DumpRegs@0 (08F10E1h)
33:
34: exit
008F369C 6A 00 push 0
008F369E EB 10 00 00 00 call _ExitProcess@4 (08F36B3h)

```


Experimental Results:

a) Rebuild & Start Debugging

CamStudio:



Completed Fully Partially : Checked by: _____

Lab 1 Submission

Due: 25 April 2024 (Thursday: 11.33pm)

This is considering as a group submission. Each group requires ONLY 1 submission.

1. Screen capture results part 1(a), (b), (c) - Start Without Debugging

2. Code Submission

i) Zip both your exercise folders;

- [Lab 1a] member 1
- [Lab 1b] member 1
- [Lab 1c] member 2

ii) Combine into ONE (1) zip file name "Lab1.zip".

iii) Upload "Lab1.zip" file to e-learning under [Labs Submission] section

2. Prepare a video demo for debugging process Lab 1(a &b) and Lab 1(c) – Start Debugging. If your group consists of 3 students, must do Lab 1(a). Each student should explain one debugging process.

Requirement for the video:

- Video duration is about 7 minutes only.
- You MUST explain verbally on how you execute this Lab 1 in the video (... show your face in recording, please 😊)
- The video can be recorded in any tools that you're comfortable
 - Example; you may use any online meeting tool like Zoom, Google Meet, MS Teams to record your lab demo.

3. Video Submission

Demo video can be submitted in two options:

a) as an .mp4 video file (to be uploaded to e-learning) – this is not preferable if the size is big.

OR

b) Upload your video to your YouTube account and share the YouTube video link in the submission option. Set your YouTube video unlisted video (i.e. only person who has the link can see the video)

- Upload OR share the YT video link to e-learning under [Lab Submission] section.

Method (b) is the most preferable.